Mathématiques numériques — Série 2

Corrigé détaillé des 8 exercices

Philippe — E2-p2, Génie électrique · HEIA-FR · Richard Baltensperger

Statut de vérification. Les tableaux et les figures proviennent d'une exécution réelle de `serie2_verification.py` : Python en arithmétique binary64, avec une référence à 80 chiffres pour les exercices 5 et 7. Les fichiers MATLAB sont fournis, mais n'ont pas été exécutés sur ce Mac, où MATLAB est absent. Les conclusions du corrigé du professeur sont distinguées des mesures Python, en particulier pour `roots` et `exp`.

Exercice 1 — Tous les positifs de Fl(2,3,−2,2)

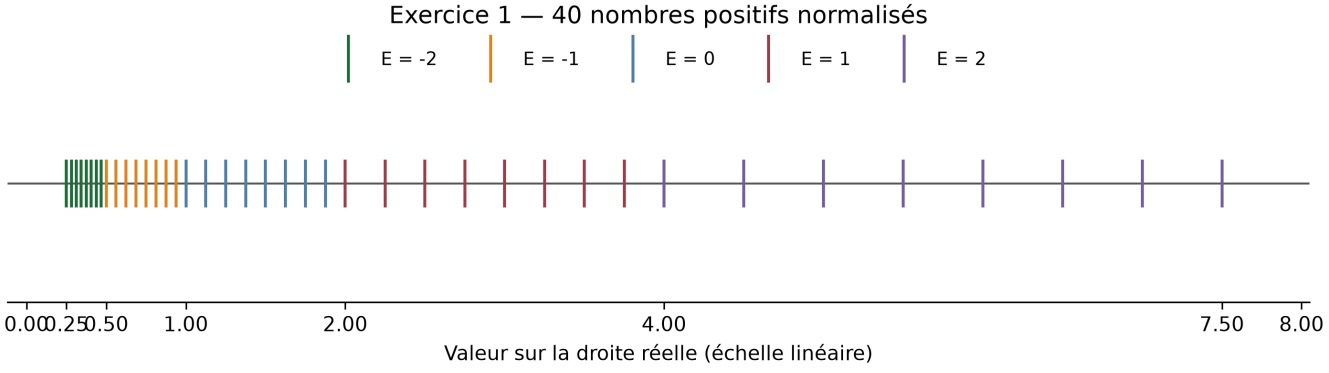
Représentation et énumération

La convention des diapositives « Les nombres réels sur l'ordinateur », pages 4–5, est : 3 bits **après** le point binaire et $-2 \le E \le 2$. Le premier bit de la mantisse est nécessairement 1.

$$x = (1.b_1b_2b_3)_2\, 2^E = \left(1 + \tfrac{j}{8}\right)2^E, \quad j = 0, \ldots, 7.$$

Il y a 8 mantisses et 5 exposants, donc **40 nombres strictement positifs normalisés**. Zéro n'en fait pas partie ; si l'on ajoutait les négatifs et zéro, on aurait 81 valeurs.

Mantisse binaire	E = −2	E = −1	E = 0	E = 1	E = 2
1.000	0,25	0,5	1	2	4
1.001	0,28125	0,5625	1,125	2,25	4,5
1.010	0,3125	0,625	1,25	2,5	5
1.011	0,34375	0,6875	1,375	2,75	5,5
1.100	0,375	0,75	1,5	3	6
1.101	0,40625	0,8125	1,625	3,25	6,5
1.110	0,4375	0,875	1,75	3,5	7
1.111	0,46875	0,9375	1,875	3,75	7,5



Droite réelle : exercice 1

À retenir. Le minimum est 1/4, le maximum est 15/2. Pour un exposant fixé, le pas vaut 2^(E−3) : 1/32, 1/16, 1/8, 1/4 puis 1/2. Les nombres sont plus serrés près de zéro ; le pas double avec l'exposant.

Exercice 2 — Mantisse sur 3(+1) bits et exposant sur 2 bits

Décoder l'exposant biaisé

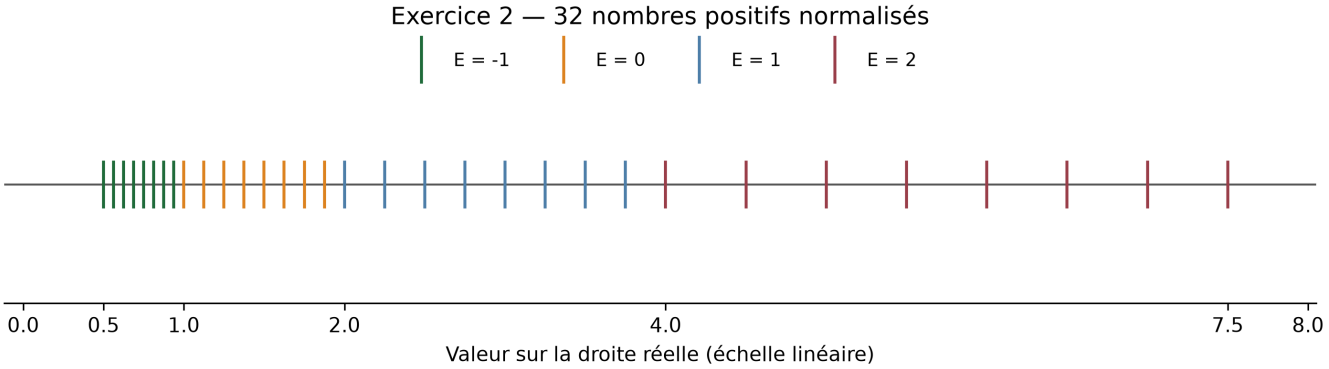
Le « +1 » désigne le premier bit 1, implicite : on stocke seulement les 3 bits de fraction. Les 2 bits d'exposant donnent e = 0, 1, 2, 3. Avec le biais d = 1 :

$$E = e - d = e - 1 \in \{-1, 0, 1, 2\}, \qquad x = \left(1 + \tfrac{j}{8}\right) 2^{e-1}.$$

Bits e	Valeur de e	Exposant réel E
00	0	−1
01	1	0
10	2	1
11	3	2

Les **32 valeurs** sont le tableau de l'exercice 1 sans sa colonne E = −2 :

Bits de fraction	Mantisse	e = 00	e = 01	e = 10	e = 11
000	1	0,5	1	2	4
001	1,125	0,5625	1,125	2,25	4,5
010	1,25	0,625	1,25	2,5	5
011	1,375	0,6875	1,375	2,75	5,5
100	1,5	0,75	1,5	3	6
101	1,625	0,8125	1,625	3,25	6,5
110	1,75	0,875	1,75	3,5	7
111	1,875	0,9375	1,875	3,75	7,5



Droite réelle : exercice 2

Vérification avec floatgui. Régler `t = 3`, `emin = -1`, `emax = 2`. Le script complet effectue ces réglages automatiquement. La figure de ce corrigé est une reproduction Python du modèle ; l'interface MATLAB reste à exécuter dans MATLAB.

Piège. Cet exercice est un modèle pédagogique, pas un petit format IEEE complet : on ne réserve pas les exposants 00 et 11 pour zéro, les sous-normaux, Inf ou NaN. L'énoncé et le corrigé du professeur utilisent les quatre exposants pour des nombres normalisés.

Exercice 3 — Compléter le nombre affiché dans floatgui.m

Comptage

Le programme construit `f = (0:2^t-1)/2^t`. Il y a donc 2^t mantisses, chacune étant multipliée par 2^E pour tous les exposants de `emin` à `emax`, bornes incluses.

$$N_+ = 2^t (e_{\max} - e_{\min} + 1).$$

Le « +1 » est indispensable : de −1 à 2 il y a quatre entiers, pas trois. Remplacer la dernière ligne par :

```
text(.9*xmax, 2, num2str(2^t*(emax-emin+1)))
```

Paramètres	Nombre de mantisses	Nombre d'exposants	Nombres positifs
t = 3, emin = −2, emax = 2	8	5	40
t = 3, emin = −1, emax = 2	8	4	32
Valeurs initiales du fichier : 3, −4, 2	8	7	56

La formule a été contrôlée par énumération et dédoublonnage sur **les 729 combinaisons entières des trois curseurs** : t = 0…8, emin = −8…0, emax = 0…8. Aucun doublon ni différence de comptage.

Fichier modifié. Une copie de travail est placée dans `scripts/serie2/floatgui.m`. Seule sa dernière expression a été changée. La version reçue dans `scripts/ncm/floatgui.m` et l'archive du professeur sont préservées. Dans la version reçue, la dernière ligne était déjà décommentée, mais affichait seulement `t`.

Pourquoi la formule du professeur fonctionne ici

Le corrigé propose `2^t*(abs(emin) + emax + 1)`. Cette expression est équivalente quand `emin ≤ 0`, ce qui est imposé par le curseur du GUI. La forme `emax-emin+1` exprime directement la longueur de l'intervalle et reste correcte même si les deux bornes sont positives.

> **À retenir.** Ce GUI compte uniquement les **positifs normalisés**, sans zéro ni sous-normaux. Pour le modèle symétrique avec les négatifs et un zéro :

$$N_{\text{total}} = 2N_+ + 1.$$

Lancer le fichier depuis le dossier scripts

```
addpath('serie2', '-begin')
floatgui
% Regler ensuite : t=3, emin=-1, emax=2.
```

Le script `serie2_exercices.m` ajoute le bon chemin et règle les curseurs pour l'exercice 2.

Exercice 4 — Pourquoi 1 − 3(4/3 − 1) n'est pas zéro

Programme demandé

```
format long
a = 4/3
b = a - 1
c = 3*b
e = 1 - c
```

En arithmétique exacte :

$$a = \frac{4}{3}, \qquad b = \frac{1}{3}, \qquad c = 1, \qquad e = 0.$$

En double précision, le calcul réel effectué en Python donne les valeurs suivantes, imprimées avec 17 chiffres significatifs :

Variable	Valeur calculée
a	1.3333333333333333
b	0.33333333333333326
c	0.99999999999999978
e	2.2204460492503131e−16

Ainsi, **le résultat calculé est epsilon machine, alors que le vrai résultat est zéro** :

$$e = 2^{-52} = \text{eps} \neq 0.$$

D'où vient l'écart ?

La fraction 4/3 a un développement binaire infini : sa représentation est arrondie. Le double stocké est légèrement inférieur à 4/3. Dans ce calcul particulier, les opérations suivantes donnent :

$$\widetilde{a} = \frac{4}{3} - \frac{2^{-52}}{3}, \quad \widetilde{b} = \frac{1}{3} - \frac{2^{-52}}{3}, \quad \widetilde{c} = 1 - 2^{-52}.$$

La soustraction finale récupère donc exactement 2^−52. Voici les décimales exactes du double de 4/3 et du produit c :

```
a = 1.33333333333333325931846502498956397175788879394531250
c = 0.99999999999999977795539507496869191527366638183593750
```

Piège. `format long` ne change pas les calculs ni la précision stockée. Il modifie seulement l'affichage ; les dernières différences peuvent être masquées par l'arrondi d'impression.

À retenir. On ne peut pas simplifier une expression algébrique comme si chaque valeur intermédiaire était exacte. Ce petit programme retrouve `eps` dans l'arithmétique utilisée ; le corrigé du professeur rappelle son emploi historique pour déterminer la précision machine.

Exercice 5 — Équation quadratique et annulation

Vérifier la formule

Pour x² + ax + b = 0, compléter le carré :

$$x^2 + ax + b = 0 \Leftrightarrow \left(x + \tfrac{a}{2}\right)^2 = \left(\tfrac{a}{2}\right)^2 - b.$$

On en déduit, quand le discriminant est positif ou nul :

$$x_{1,2} = -\tfrac{a}{2} \pm \sqrt{\left(\tfrac{a}{2}\right)^2 - b}\,.$$

En développant $(x-x_1)(x-x_2)$, les relations de Viète sont :

$$x_1 + x_2 = -a, \qquad x_1 x_2 = b.$$

Coquille dans l'énoncé. Il imprime `x1 + x2 = a`. Pour le polynôme x² + ax + b, le signe correct est **−a**.

(a) Programme utilisant directement la formule

Ici a = −100000000 et b = 1. Les deux racines sont réelles et positives.

```
function [x1, x2] = solution_equation_a(a, b)
    x1 = -a/2 + sqrt((a/2)^2 - b);
    x2 = -a/2 - sqrt((a/2)^2 - b);
end
```

(b) Programme utilisant une racine et Viète

On conserve la grande racine, obtenue par addition de termes de même signe, puis on déduit la petite racine par une division :

```
function [x1, x2] = solution_equation_b(a, b)
    x1 = -a/2 + sqrt((a/2)^2 - b);
    x2 = b/x1;
end
```

La réécriture de la petite racine vient aussi de la rationalisation :

$$x_2 = \frac{b}{-a/2 + \sqrt{(a/2)^2 - b}}.$$

Pourquoi (a) échoue. La petite racine est la différence de deux nombres très proches de 50 millions. Leurs erreurs d'arrondi deviennent importantes relativement à un résultat de l'ordre de 10^−8 : c'est une **annulation catastrophique**. La méthode (b) évite cette soustraction.

Domaine de cette version (b). La branche « + » est la bonne pour cet exemple où a < 0. Pour un a général, il faut choisir la racine calculée sans annulation (la branche dépend du signe de a), puis obtenir l'autre par le produit de Viète.

Exercice 5 — Vérifications et comparaison avec roots

Valeurs de référence et résultats mesurés

Les racines exactes, calculées à 80 chiffres, ne sont **pas exactement** 10^8 et 10^−8 :

```
x_grande = 99999999.99999998999999999999998999999999999998...
x_petite = 0.00000001000000000000000100000000000000200000000...
```

Méthode exécutée en Python	Grande racine	Petite racine	Erreur relative petite
(a) formule directe	100000000	7.4505805969238281e−9	0.2549419403
(b) Viète	100000000	1e−8	7.9077e−17
NumPy roots (contrôle indépendant)	99999999.999999985	1.0000000000000002e−8	8.6359e−17

La petite racine de (a) présente donc environ **25,49 % d'erreur**, contre une erreur de l'ordre de la précision machine avec (b).

Vérification par substitution et par Viète

On calcule P(x) = x² − 10^8 x + 1, puis le produit et la somme des racines. Les résidus ci-dessous sont ceux de l'évaluation directe en double :

Méthode	P(grande)	P(petite)	Produit des racines
(a)	1	0.2549419403076172	0.7450580596923828
(b)	1	1.1102230246251565e−16	1

Le produit de (a) devrait valoir 1 : il confirme l'erreur de la petite racine. Pour (b), la somme calculée est 100000000.00000001, proche de −a = 100000000 à l'échelle de ces nombres.

> **Piège du résidu absolu.** P(10^8) = 1, mais les deux grands termes valent 10^16. Un résidu égal à 1 ne signifie donc pas une erreur relative importante sur cette grande racine. Le résidu normalisé vaut :

$$\eta(x) = \frac{|P(x)|}{|x|^2 + |a|\,|x| + |b|}; \qquad \eta(10^8) \simeq 5\,10^{-17}.$$

Pour la petite racine, le résidu normalisé est environ 0,146 avec (a), contre 3,95·10^−17 avec (b), en évaluant P à 80 chiffres sur les doubles obtenus. L'évaluation du résidu en double peut elle-même subir une annulation.

Comparaison MATLAB demandée

```
a = -100000000; b = 1;
[x1a, x2a] = solution_equation_a(a, b);
[x1b, x2b] = solution_equation_b(a, b);
r = sort(roots([1 a b]), 'descend');
disp([x1a x2a; x1b x2b; r.'])
```

Conclusion du professeur. Sa solution indique que (b) donne de meilleurs résultats que (a) et retrouve la solution de `roots`. La commande est incluse dans le script livré, mais sa sortie MATLAB n'est pas prétendue mesurée ici. Le contrôle `numpy.roots` ci-dessus est distinct : il donne lui aussi des racines précises, avec des derniers bits différents de (b).

Exercice 6 — Les trois boucles et les limites du double

1. Addition d'un incrément de plus en plus petit

```
x = 1; p = 0; y = 1; z = x + y;
while x ~= z
    y = y/2;
    p = p + 1;
    z = x + y;
end
disp(1 + 1/2^p)
disp(p)
```

Résultat : 1 puis 53. Après p passages, y = 2^−p. À p = 52, 1 + 2^−52 est le double qui suit 1. À p = 53, la somme exacte est au milieu de ces deux doubles ; l'arrondi au plus proche, avec égalité départagée vers le bit pair, donne 1. La boucle trouve le **premier incrément perdu**, pas le dernier incrément visible : `eps = 2^-(p-1)`.

2. Division jusqu'à zéro

```
x = 1; q = 0;
while x > 0
    x = x/2;
    q = q + 1;
end
disp(1/2^q)
disp(q)
```

Résultat : 0 puis 1075. Les nombres sous-normaux permettent de descendre sous `realmin = 2^-1022`. À q = 1074, x = 2^−1074 est encore positif, soit environ 4,94·10^−324. À la division suivante, 2^−1075 est arrondi à zéro. Attention : dans l'affichage `1/2^q`, le dénominateur `2^1075` déborde déjà vers Inf, donnant aussi 0 ; on vérifie le dernier x positif avec `2^-(q-1)`, pas `1/2^(q-1)`.

3. Multiplication jusqu'à Inf

```
x = 1; r = 0;
while x ~= inf
    x = 2*x;
    r = r + 1;
end
disp(2^r)
disp(r)
```

Résultat : Inf puis 1024. La valeur 2^1023 est finie, mais 2^1024 déborde. Cette boucle repère le premier débordement parmi les puissances de 2 ; elle ne calcule pas directement le plus grand double fini :

$$\text{realmax} = (2 - 2^{-52})\,2^{1023} \simeq 1.7976931348623157\,10^{308}.$$

Vérification. Les trois comptes 53, 1075 et 1024 ont été retrouvés par exécution des boucles en Python et concordent avec le corrigé du professeur. `realmin` est le minimum positif **normalisé**, pas le minimum positif tout court.

Exercice 7 — Approximer exp(x) par Taylor

Récurrence pour éviter puissances et factorielles

$$T_n(x) = \sum_{k=0}^{n} \frac{x^k}{k!}, \qquad t_0 = 1, \quad t_k = t_{k-1}\frac{x}{k}.$$

En effet, si t_(k−1) = x^(k−1)/(k−1)!, alors t_(k−1)·x/k = x^k/k!. On additionne les termes depuis k = 0 jusqu'à n inclus : l'approximation contient **n+1 termes**.

Fonction mon_exp.m

```
function y = mon_exp(x, n)
    % x scalaire ; n entier positif ou nul.
    terme = 1;
    y = 1;
    for k = 1:n
        terme = terme*x/k;
        y = y + terme;
    end
end
```

Le fichier livré ajoute une validation des arguments. Pour x = 0, la fonction rend 1 ; pour n = 0, elle rend également 1, quel que soit x. Aucun calcul de factorielle ni de puissance n'est nécessaire.

Script driver_exp.m : les 50 points et les quatre sous-graphes

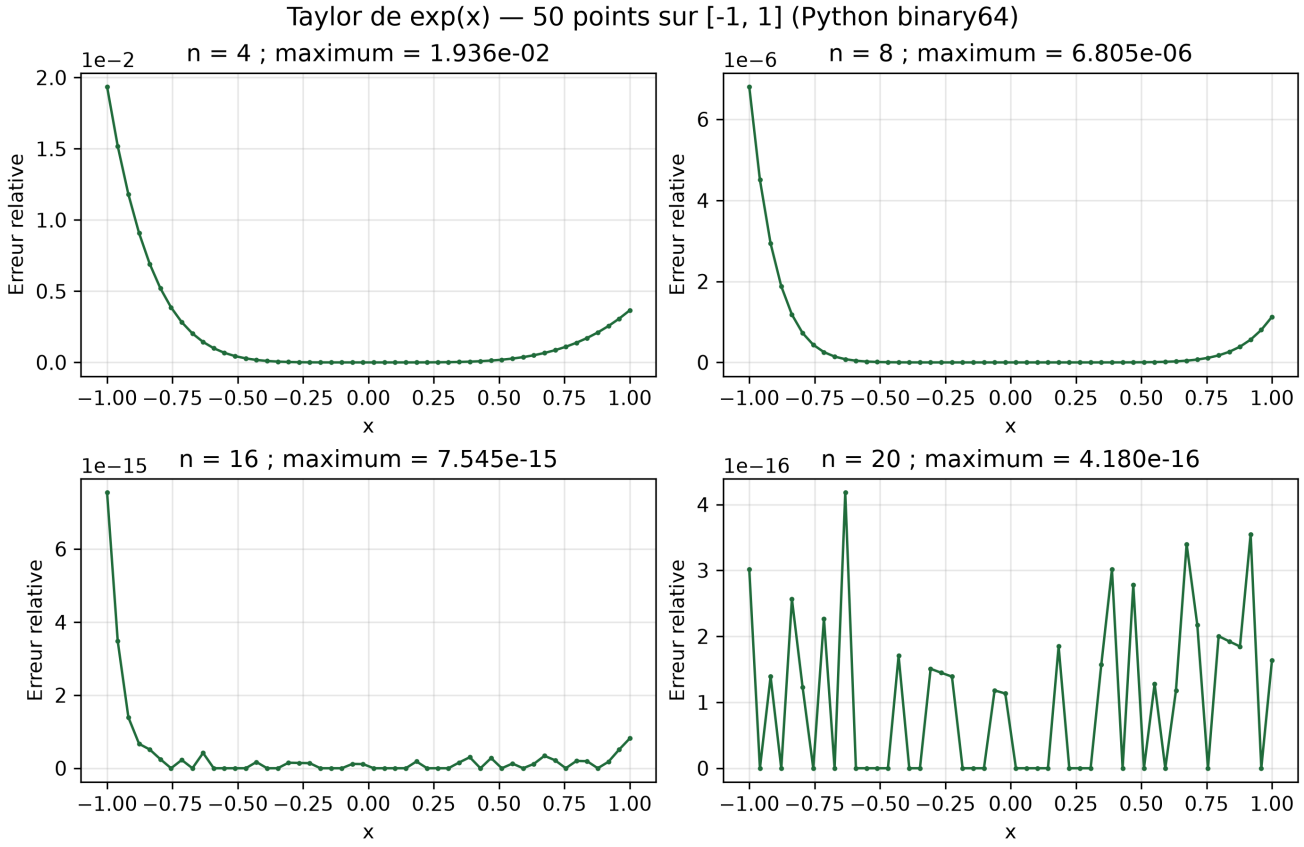
```
m = 50;
x = linspace(-1, 1, m);
reference = exp(x);
y = zeros(1, m);
k = 0;
figure
for n = [4 8 16 20]
    for i = 1:m
        y(i) = mon_exp(x(i), n);
    end
    ErrRel = abs(y - reference)./abs(reference);
    k = k + 1;
    subplot(2, 2, k)
    plot(x, ErrRel, '.-')
    grid on
    xlabel('x'); ylabel('Erreur relative')
    title(sprintf('n = %d', n))
end
```

Piège MATLAB. L'erreur relative est un calcul terme à terme : utiliser `./`, pas `/`. `linspace(-1,1,50)` inclut les deux bornes, mais pas zéro, car le nombre de points est pair. Les tests de la fonction à x = 0 ont donc été effectués séparément.

Exercice 7 — Courbes et interprétation des erreurs

Erreur relative mesurée

$$\mathrm{ErrRel}(x) = \frac{|T_n(x) - \exp(x)|}{|\exp(x)|}.$$



Les quatre erreurs de Taylor

n	Maximum mesuré en double	Maximum de troncature à 80 chiffres
4	1.9355685672e−2	1.9355685672e−2
8	6.8046015135e−6	6.8046015133e−6
16	7.5447476834e−15	7.2390386917e−15
20	4.1802086435e−16	5.0887306742e−20

Interprétation. De n = 4 à 16, ajouter des termes réduit nettement l'erreur de troncature. À n = 20, cette erreur mathématique est devenue bien inférieure à la précision machine : le graphique est dominé par les arrondis des produits, divisions, additions et de la fonction de référence. Augmenter encore n ne garantit donc pas un résultat plus précis en double.

Pour n = 4, 8 et 16, les maxima mesurés sont atteints à x = −1. Pour n = 20, le maximum mesuré est à x ≈ −0,632653 : sa position dépend des derniers bits d'arrondi.

Statut de la figure. Les 50 abscisses et la récurrence sont celles demandées. Les courbes ont été générées réellement en Python binary64 à 300 dpi. La fonction `exp` de MATLAB et ses derniers bits peuvent différer : les valeurs proches de 10^−16 ne sont pas une sortie MATLAB reproduite.

Exercice 8 — Dix additions de 0,1

(a) Programme et constat

```
somme = 0;
for i = 1:10
    somme = somme + 0.1;
end
if somme == 1
    disp('Le compte est bon')
else
    disp('Detournement de fonds!')
end
```

L'exécution des dix additions donne :

```
somme     = 0.99999999999999989
somme - 1 = -1.1102230246251565e-16
somme == 1 : faux
```

Le programme affiche donc « **Détournement de fonds!** », même si dix dixièmes valent mathématiquement 1. La fraction 1/10 possède une écriture binaire infinie ; chaque opération s'effectue sur des valeurs arrondies. Le double représentant 0,1 est exactement :

```
0.1000000000000000055511151231257827021181583404541015625
```

> **Piège.** Arrondir l'affichage pour montrer 1 ne rend pas `somme == 1` vrai. Et le produit `10*0.1` n'est pas la même suite d'opérations que dix additions : un résultat peut valoir exactement 1 en double sans que l'autre le vaille.

(b) Modifier la comparaison, pas seulement l'affichage

La solution proposée par le professeur utilise une tolérance absolue :

```
if abs(somme - 1) <= 1e-5
    disp('Le compte est bon')
else
    disp('Detournement de fonds!')
end
```

L'écart mesuré est bien inférieur à 10^−5 : le message devient « **Le compte est bon** ». Pour ce calcul très simple à l'échelle de 1, une tolérance `10*eps` est aussi suffisante ; elle ne doit pas devenir une règle aveugle pour tous les problèmes. La tolérance doit refléter l'échelle des nombres et l'erreur acceptable.

Autre approche pour des montants en dixièmes ou en centimes. Stocker et additionner des unités entières, puis convertir à l'affichage. On ne fait alors pas dépendre un contrôle comptable de l'égalité de deux résultats flottants.

Sources et reproduction

Sources locales : `Consignes/sources/matnum-serie2-exercices.pdf` (2 p.), `matnum-serie2-solutions.pdf` (1 p.), diapositives « Les nombres réels sur l'ordinateur » (p. 4–7) et `scripts/ncm/floatgui.m`.

MATLAB : ouvrir `scripts/serie2_exercices.m` et exécuter le script complet ou ses sections. Les fonctions de l'exercice 5 sont définies à sa fin ; `mon_exp.m` et `driver_exp.m` sont des fichiers séparés.

Reproduction des résultats et du PDF depuis la racine du dépôt :

```
.venv/bin/python Semestre-3/E2-MATNUM/scripts/test_serie2_verification.py
.venv/bin/python Semestre-3/E2-MATNUM/scripts/serie2_verification.py
.venv/bin/python Semestre-3/E2-MATNUM/scripts/serie2_corrige_pdf.py
```